Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

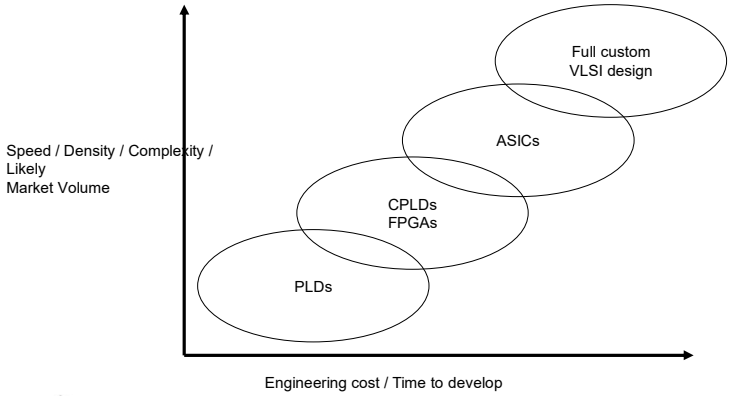
Lojik Devreler ve Tasarım

Bahar 2023

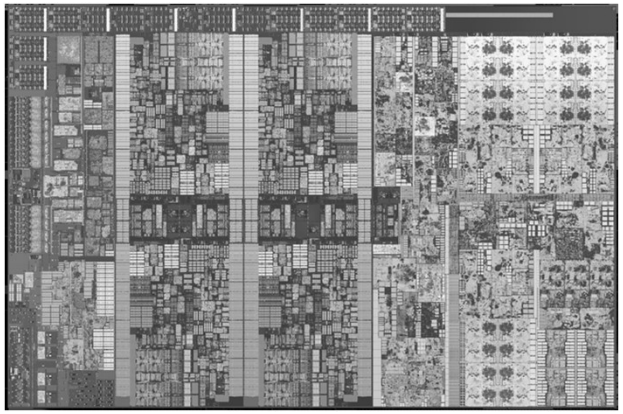
Dr. Öğr. Ü. İhsan Çiçek

8. Donanım Tanımlama Dilleri: Birleşik Lojik Devreler ve Test Tezgahları

DIGITAL DESIGN Technology Tradeoffs



Intel Kaby Lake, 7th Gen, 14nm, 2017



https://en.wikichip.org/wiki/intel/microarchitectures/kaby_lake

How to manage design complexity?

Hardware description languages (HDLs)

- Specify complex designs
- Keep track of changes and manage within teams
- Produce synthesizable code
- Simulate (easily) and verify for functional and timing operation

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

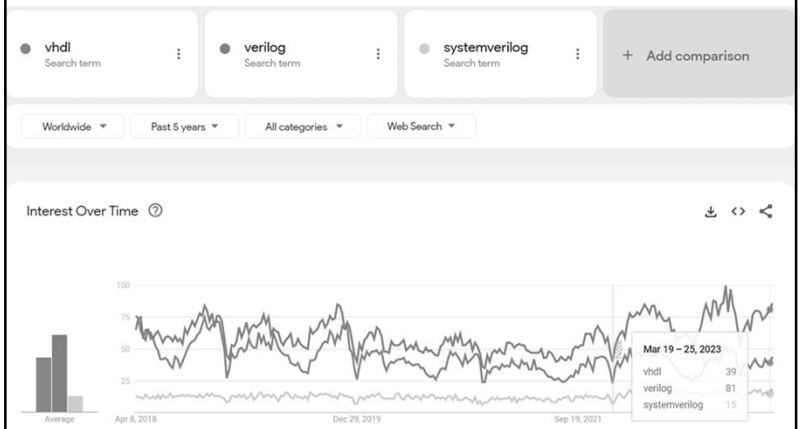
Hardware Description Languages

There are two well-known ones

- VHDL (VHSIC Hardware Description Language)
  - 1981 by US DoD, 1987 IEEE standard - **IEEE 1076-1987**
  - 1993, 2002, 2008, 2019 Updates to standard (**IEEE 1076-2019**)
- Verilog
  - 1984 by Gateway Design Automation
  - 1995 IEEE standard - **IEEE 1364-1995**
  - 2001, 2005 Updates to standard (**1364-2005**)
- SystemVerilog
  - Superset of Verilog
  - 2002 by Co-Design Automation
  - 2005 IEEE standard - IEEE 1800-2005
  - 2009 Merged with Verilog standard - **IEEE 1800-2009**
  - 2017 Update to the standard (**IEEE 1800-2017**)

Most commercial designs built using HDLs!!!

trends.google.com - April 2023

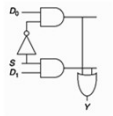


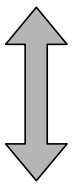
vhdl, verilog, systemverilog - Explore - Google Trends

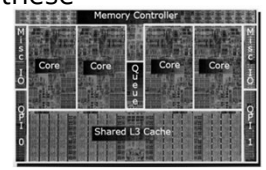
Hierarchical Design

How can we go from a single gate to a complex IC?

- We need to stop the **abstraction** at some level, and this level is usually **logic gates**.
- Define **modules** out of these gates such as *a MUX*.
- Define bigger components out of these modules such as *an ALU*.







How can we approach the design?

- Top-Down Design Methodology
- Bottom-Up Design Methodology

Design Methodologies

Top-Down Design Methodology

- Define **top-level** module and identify its building blocks as **sub-modules**
- Define sub-modules, and their sub-modules, .. until the smallest unit is defined.
- Smallest unit is called a **leaf-cell**. This can be a combination of logic cells, or a technology dependent cell library.

Bottom-Up Design Methodology

- Define the **basic building blocks**. (leaf-cells)
- Using these building blocks, create **bigger modules.** (sub-modules)
- Using these modules, create a **top-level module**.

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Module in SystemVerilog

A **module** is the basic building block in **SystemVerilog.**



It requires <u>three</u> properties:

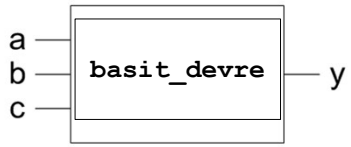
1. Module name (e.g. example)
2. Port names (e.g. a, b, c, y)
3. Port directions (e.g. input / output)

After the module definition, we define the **functionality** of the module.

1. **Behavioral** describe what a module does
2. **Structural** describe how it is built from simpler modules

Module Definition in SystemVerilog

- start with **module**, and end with **endmodule** keywords
- **logic** keyword is necessary to define the type of the signal



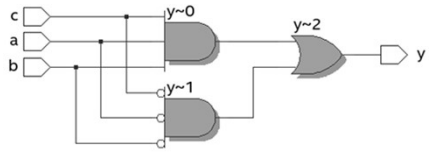

```
/* basit_devre.sv
 * SystemVerilog basit bir devre örneği
 */
module basit_devre (
       input  logic a, b, c,
       output logic y
);
// Devre tanımı buraya gider

endmodule
```

<u>Three</u> properties

1. **Module name**
2. **Port names**
3. **Port directions**

Behavioral design



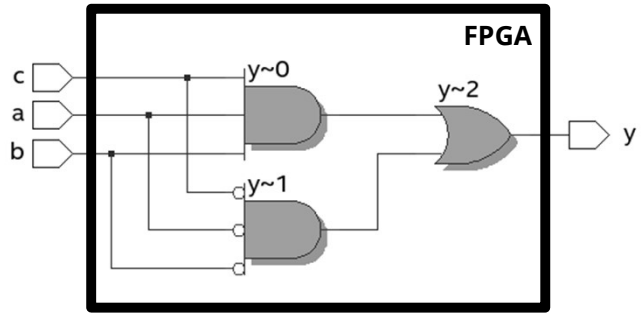

```
/* basit_devre.sv
 * SystemVerilog basit bir devre örneği
 */
module basit_devre (
       input  logic a, b, c,
       output logic y
);
assign y = a & b & c | ~a & ~b & ~c;

endmodule
```

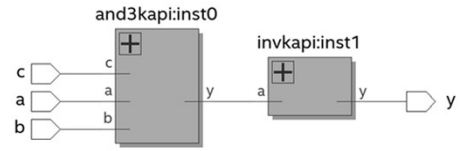
assign keyword is used to define the combinational logic assignments.

FPGA view of the circuit

Here is how the FPGA will look like after synthesize & **fitting** and programming the board.



Structural Design




```
module invkapi
(
input  logic a,
output logic y
);

assign y = ~a;

endmodule
```

```
module and3kapi (
input  logic a, b,
c,
output logic y
);

assign y = a & b &
c;

endmodule
```

```
module nand3kapi (
input  logic a, b, c,
output logic y
);

logic n1;

and3kapi inst0(a, b, c,
n1);
invkapi  inst1(.a(n1),
.y(y));

endmodule
```


Structural Design

Two ways to instantiate.

- **Implicit** (map ports based on **order**)
- **Explicit** (map ports based on **port name**)

Implicit instantiation is easier to make **mistakes**. Go with **explicit**.

```
module and3kapi (
input  logic a, b, c,
output logic y );
```

```
module invkapi(
input  logic a,
output logic y);
```

```
module nand3kapi (
   input  logic a, b, c,
   output logic y
);
logic n1;
and3kapi inst0(.a(a), .b(b), .c(c),
.y(n1));
invkapi  inst1(.a(n1), .y(y));
endmodule
```

```
module nand3kapi (
input  logic a, b, c,
output logic y
);
logic n1;
and3kapi inst0(a, b, c,
n1);
invkapi  inst1(n1, y);
endmodule
```


SystemVerilog Basic Syntax

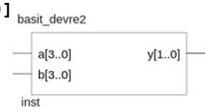
- SystemVerilog is **Case sensitive**
  - **reset** and **Reset** are not the same signal.
- Names cannot start with numbers
  - **2mux** is an invalid name
- **Whitespace**s are ignored
- C style comments:
  - `// single line comment`
  - `/* multiline comment */`

How about multi-bit ports?

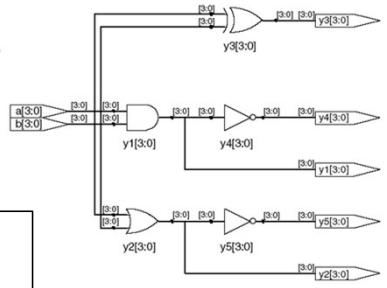
- It becomes difficult to keep track of all the port names, so we **group** the same purpose bits into a single **multi-bit** port, also called **a bus**.
- Before the port names, define the range between **[ ]** with the syntax **`[range_end:range_start]`**
  - Total number of bits: **`range_end - range_start + 1`**

```
module basit_devre2 (
      input  logic [3:0] a, b // a[3], a[2], a[1], a[0],
b[3], ... , b[0]
      output logic [1:0] y    // y[1], y[0]
);

endmodule
```



Bitwise Operators



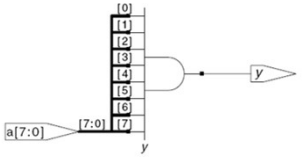

```
module gates (
input  logic [3:0]  a, b,
output logic [3:0] y1, y2, y3,
y4, y5
);
   assign y1 = a & b;    // AND
   assign y2 = a | b;    // OR
   assign y3 = a ^ b;    // XOR
   assign y4 = ~(a & b); // NAND
   assign y5 = ~(a | b); // NOR
endmodule
```


Reduction Operators

```
module and8 (
input  logic [7:0] a,
output logic       y );

   assign y = &a;
   // &a is much easier to write than
   // assign y = a[7] & a[6] & a[5] & a[4] &
   //            a[3] & a[2] & a[1] & a[0];

endmodule
```



It is also applicable to OR and XOR gates.

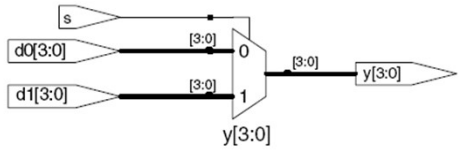
Conditional Assignments

```
module mux2 (
    input  logic [3:0] d0, d1,
    input  logic       s,
    output logic [3:0] y
);

   assign y = s ? d1 : d0;
   // if s is 1, then y = d1, otherwise y = d0

endmodule
```

- **?** is also called a **ternary operator** since it operates on <u>three</u> inputs



SystemVerilog operators

`&` - Bitwise AND	`<` - Less than
`|` - Bitwise OR	`<=` - Less or equal
`~` - Bitwise negation (NOT)	`>` - Greater than
`^` - Bitwise XOR	`>=` - Greater or equal
`||` - Logical OR	
`&&` - Logical AND	`+` - Addition
`!` - Logical negation	`-` - Subtraction
`<<` - Left Shift	
`>>` - Right Shift	`*` - Multiplication
`==` - Equality	`/` - Division
`!=` - Inequality	`%` - Remainder

FPGAs usually come with fast carry chains to implement these efficiently.

Careful with these. Since we describe hardware, the resulting logic might be huge.

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

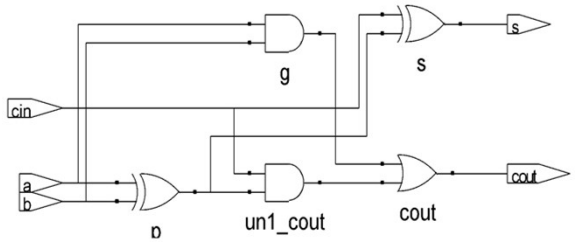
Internal Variables

```
module fulladder (input  logic a, b, cin,
                  output logic s, cout);
  logic p, g;   // internal nodes

  assign p = a ^ b;
  assign g = a & b;

  assign s = p ^ cin;
  assign cout = g | (p & cin);
endmodule
```

Q. What will be the **order of** the assignments? (or which **assign** will happen first?)



Precedence

Highest	~	NOT
	*, /, %	mult, div, mod
	+, -	add,sub
	<<, >>	shift
	<<<, >>>	arithmetic shift
	<, <=, >, >=	comparison
	==, !=	equal, not equal
	&, ~&	AND, NAND
	^, ~^	XOR, XNOR
	|, ~|	OR, NOR
Lowest	?:	ternary operator

Number Representations

N'Bxx

N = number of bits, **B** = base, **xx** = value
N'B is optional but recommended (**default is decimal**)

Number	# Bits	Base	Decimal Equivalent	Stored
3'b101	3	binary	5	101
'b11	unsized	binary	3	00…0011
8'b11	8	binary	3	00000011
8'b1010_1011	8	binary	171	10101011
3'd6	3	decimal	6	110
6'o42	6	octal	34	100010
8'hAB	8	hexadecimal	171	10101011
42	unsized	decimal	42	00…0101010

Bit Manipulations

- By **slicing**, **concatenating** and **duplicating** signals, we can create **a new** multi-bit signals.

```
assign y = {a[2:1], {3{b[0]}}, a[0], 6'b100_010};

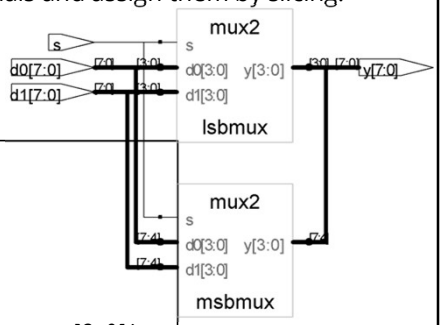
// if y is a 12-bit signal (y[11:0]), the above statement
produces:
//    y = a[2] a[1] b[0] b[0] b[0] a[0] 1 0 0 0 1 0

//    i.e. y[11] = a[2], y[10] = a[1], y[9] = b[0], ...,
y[1] = 1, y[0] = 0

// underscores (_) are used for formatting only to make
// it easier to read. SystemVerilog ignores them.
```

Bit Manipulations II

- We can also divide the signals and assign them by slicing.




```
module mux2_8 (
input  logic [7:0] d0, d1,
input  logic       s,
output logic [7:0] y
);

mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);
mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);

endmodule
```

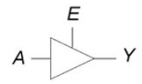

Z: Floating Output

- We can also use **Z** state for a tristate buffer operation.

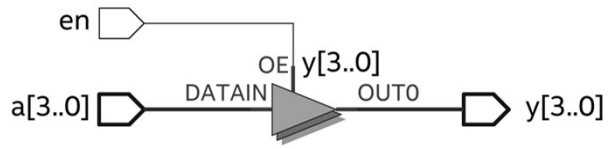
```
module tristate (input  logic [3:0] a,
                 input  logic       en,
                 output logic [3:0] y);

   assign y = en ? a : 4'bz;

endmodule
```



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1



Parameterized Modules

- What if we want to instantiate multiple, multi-port circuit with different port numbers?

```
module mux2
    #(parameter width = 8)  // name and default value
    (input  logic [width-1:0] d0, d1,
     input  logic             s,
     output logic [width-1:0] y);

  assign y = s ? d1 : d0;
endmodule
```

```
module myawesomecircuit (
...
// Instance with 8-bit bus width (uses default):
  mux2 inst0(d0, d1, s, out);

// Instance with 12-bit bus width:
  mux2 #(12) inst1(d0, d1, s, out);
...
endmodule
```


Delay

- We can specify delay on SystemVerilog using the **#number** syntax and giving **a time unit**

```
`timescale 1ns/1ps

module basit_devre (input logic a, output logic b);
logic n1;
assign #4 n1 = a & b; // assign a and b after 4ns to n1
assign #3 b = ~n1; // assign inverted n1 after 3ns to b

endmodule
```

- Delay is **only** for **simulation**
- **Not synthesizable**
- Used for delay modeling in circuits

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

HDL to Gates

Synthesis

- Transforms HDL code into a **netlist** describing the hardware (i.e., a list of gates and the wires connecting them)
- Tools **map** these netlists into FPGA hardware.
- Not always possible or optimal.

Simulation

- Tests the circuit operation **without** manufacturing
- Inputs applied to circuit
- Outputs checked for correctness
- Millions of dollars saved by debugging in **simulation** instead of hardware

HDL Tips

- Define a naming scheme and stick to it.
  - Eg.: **i_a, i_b, o_c** or **in_a, ou_b** or **a_i, b_o**
- While defining a bus use **MSB:LSB** order, more intuitive.
  - **Eg.:** instead of **a[0:31]** use **a[31:0]**.
- Create only one module in one file. Easier to organize.
- Make the name of the module and filename the same.
  - Eg.: **alu_devresi.sv** file has **alu_devresi** module.
- **Don't forget** HDL is used **to define and construct hardware**. We describe **how the hardware is connected** and **how it operates**.
  - Don't confuse it with programming languages.

Simulation

- Devrenin doğru çalışıp çalışmadığını test etmek için, test edilen devrenin girişlerine farklı sinyaller ve örnekler göndermek ve çıkışları gözlemek için kurulan devreye **testbench** denir.
  - Ayrı bir devre - genelde port barındırmaz.
  - Test edilen devreyi çağırıp, **port bağlaması** yapılır. Test edilen devreye **design under test (dut)** veya **unit under test (uut)** adı verilir.
- Bu testbench devresi **sentezlenemez**.
- Test metodolojisine göre basit, veya otomatik kontrole dayalı kullanılabilir.
- **İyi** bir **testbench**, devredeki **hataları** bulmak için **büyük önem** taşır.

Simulation

- Commercial simulators
  - **ModelSim and Questa** (Mentor Graphics)
  - **Incisive Enterprise Simulator** (Cadence)
  - **VCS** (Synopsys)
  - Active-HDL/Riviera-PRO (Aldec)
  - Vivado Simulator (Xilinx)
- Open-source simulators
  - Icarus Verilog
  - Verilator
  - GHDL
- We will use **ModelSim Quartus Edition** from **Intel**

Example testbench circuit

```
// 1ns birim zaman, ve 1ps çözünürlükle çalıştır
`timescale 1ns/1ps

module tb_basit_devre ();
    // basit_devre ye bağlanacak giriş ve cıkışlar
    logic a, b, c;
    logic y;

    // test edeceğimiz devreyi bağlayalım
    basit_devre dut0(a, b, c, y);

    // girişleri, 10 birim zaman aralıklarla değiştir
        // initial blokunun içine test paternlerini ekleyebiliriz
    initial begin
        a = 0; b = 0; c = 0; #10; // a, b ve c yi gönder ve 10 ns bekle
        c = 1;               #10; // sadece c yi değiştir ve 10 ns bekle
        a = 1; c = 0;        #10;
        b = 1; c = 1;        #10;

        $stop;                    // simülasyon bittiğinde sonlandir
    end
endmodule
```


Example testbench - autocontrol

```
// 1ns birim zaman, ve 1ps çözünürlükle çalıştır
`timescale 1ns/1ps

module tb_basit_devre ();
    ...
    // test edeceğimiz devreyi bağlayalım
    basit_devre dut0(a, b, c, y);
        ...
    initial begin
        a = 0; b = 0; c = 0; #10;
        if (y !== 1) $display("000 failed.");
        c = 1;               #10;
        if (y !== 0) $display("001 failed.");
        a = 1; c = 0;        #10;
        if (y !== 0) $display("100 failed.");
        b = 1; c = 1;        #10;
        if (y !== 1) $display("111 failed.");

        $stop;                    // simülasyon bittiğinde sonlandir
    end
endmodule
```


Testbenches

What Will We Learn?

- How to simulate your circuit
- Applying inputs
- Seeing if the circuit does the correct thing

Testbenches

How Do You Know That A Circuit Works?

- You have written the Verilog code of a circuit
  - Does it work correctly?
  - Even if the syntax is correct, it might do what you want?
  - What exactly it is that you want anyway?
- Trial and error can be costly
  - You need to 'test' your circuit in advance
- In modern digital designs, functional verification is the most time consuming design stage.

Testbenches

The Idea Behind A Testbench

- **Using a computer simulator to test your circuit**
  - You instantiate your design
  - Supply the circuit with some inputs
  - See what it does
  - Does it return the "correct" outputs?

Testbenches

Testbenches

- **HDL code written to test another HDL module, the *device under test* (dut), also called the *unit under test* (uut)**
- **Not synthesizeable**
- **Types of testbenches:**
  - Simple testbench
  - Self-checking testbench
  - Self-checking testbench with testvectors

Testbenches

Example

- **Write Verilog code to implement the following function in hardware:**

$$y = (\overline{b} \cdot \overline{c}) + (a \cdot \overline{b})$$

- **Name the module sillyfunction**

Testbenches

Example

- **Write Verilog code to implement the following function in hardware:**

$$y = (\overline{b} \cdot \overline{c}) + (a \cdot \overline{b})$$

- **Name the module sillyfunction**

```
module sillyfunction(input  a, b, c,
                     output y);

  assign y = ~b & ~c | a & ~b;
endmodule
```

Testbenches

Simple Testbench

```
module testbench1(); // Testbench has no inputs, outputs
  reg  a, b, c;      // Will be assigned in initial block
  wire y;

  // instantiate device under test
  sillyfunction dut (.a(a), .b(b), .c(c), .y(y) );d

  // apply inputs one at a time
  initial begin               // sequential block
    a = 0; b = 0; c = 0; #10; // apply inputs, wait 10ns
    c = 1; #10;               // apply inputs, wait 10ns
    b = 1; c = 0; #10;        // etc .. etc..
    c = 1; #10;
    a = 1; b = 0; c = 0; #10;
  end
endmodule
```


Testbenches

Simple Testbench

- Simple testbench instantiates the design under test
- It applies a series of inputs
- The outputs have to be observed and compared using a simulator program.
  - This type of testbench does not help with the outputs
- `initial` statement is similar to `always`, it just starts once at the beginning, and does not repeat.
- The statements have to be blocking.

Testbenches

Self-checking Testbench

```
module testbench2();
  reg  a, b, c;
  wire y;

  // instantiate device under test
  sillyfunction dut(.a(a), .b(b), .c(c), .y(y));

  // apply inputs one at a time
  initial begin
     a = 0; b = 0; c = 0; #10; // apply input, wait
     if (y !== 1) $display("000 failed."); // check
     c = 1; #10;               // apply input, wait
     if (y !== 0) $display("001 failed."); // check
     b = 1; c = 0; #10;        // etc.. etc..
     if (y !== 0) $display("010 failed."); // check
  end
endmodule
```


Testbenches

Self-checking Testbench

- Better than simple testbench
- This testbench also includes a statement to check the current state
- `$display` will write a message in the simulator
- This is a lot of work
  - Imagine a 32-bit processor executing a program (thousands of clock cycles)
- You make the same amount of mistakes when writing testbenches as you do writing actual code

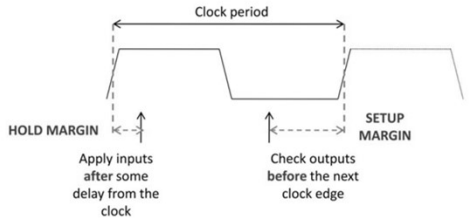
Testbenches

Testbench with Testvectors

- The more elaborate testbench
- Write testvector file: inputs and expected outputs
  - Usually can use a high-level model (golden model) to produce the 'correct' input output vectors
- Testbench:
  - Generate clock for assigning inputs, reading outputs
  - Read testvectors file into array
  - Assign inputs, get expected outputs from DUT
  - Compare outputs to expected outputs and report errors

Testbenches

Testbench with Testvectors



- A testbench clock is used to synchronize I/O
  - The same clock can be used for the DUT clock
- Inputs are applied following a hold margin
- Outputs are sampled before the next clock edge
  - The example in book uses the falling clock edge to sample

Testbenches

Testvectors File

- We need to generate a testvector file (somehow)
- File: example.tv – contains vectors of abc_yexpected

```
000_1
001_0
010_0
011_0
100_1
101_1
110_0
111_0
```


Testbenches

Testbench: 1. Generate Clock

```
module testbench3();
  reg         clk, reset;    // clock and reset are internal
  reg         a, b, c, yexpected;  // values from testvectors
  wire        y;                   // output of circuit
  reg  [31:0] vectornum, errors;   // bookkeeping variables
  reg  [3:0]  testvectors[10000:0];// array of testvectors

  // instantiate device under test
  sillyfunction dut(.a(a), .b(b), .c(c), .y(y) );

  // generate clock
  always     // no sensitivity list, so it always executes
    begin
      clk = 1; #5; clk = 0; #5;     // 10ns period
    end
```

Testbenches

2. Read Testvectors into Array

```
// at start of test, load vectors
// and pulse reset

 initial             // Will execute at the beginning once
    begin
      $readmemb("example.tv", testvectors); // Read vectors
      vectornum = 0; errors = 0;            // Initialize
      reset = 1; #27; reset = 0;       // Apply reset wait
    end


// Note: $readmemh reads testvector files written in
// hexadecimal
```


Testbenches

3. Assign Inputs and Expected Outputs

```
// apply test vectors on rising edge of clk
   always @(posedge clk)
     begin
       #1; {a, b, c, yexpected} = testvectors[vectornum];
     end
```

- **Apply inputs with some delay (1ns) AFTER clock**
- **This is important**
  - Inputs should not change at the same time with clock
- **Ideal circuits (HDL code) are immune, but real circuits (netlists) may suffer from hold violations.**

Testbenches

4. Compare Outputs with Expected Outputs

```
// check results on falling edge of clk
   always @(negedge clk)
    if (~reset)               // skip during reset
      begin
      if (y !== yexpected)
      begin
        $display("Error: inputs = %b", {a, b, c});
        $display("  outputs = %b (%b exp)",y,yexpected);
        errors = errors + 1;
      end
// Note: to print in hexadecimal, use %h. For example,
//       $display("Error: inputs = %h", {a, b, c});
```


Testbenches

4. Compare Outputs with Expected Outputs

```
// increment array index and read next testvector
      vectornum = vectornum + 1;
      if (testvectors[vectornum] === 4'bx)
      begin
          $display("%d tests completed with %d errors",
                    vectornum, errors);
          $finish;                  // End simulation
      end
    end
endmodule

// Note: === and !== can compare values that are
// x or z.
```

Testbenches

Golden Models

- **A golden model represents the ideal behavior of your circuit.**
  - Still it has to be developed
  - It is difficult to get it right (bugs in the golden model!)
  - Can be done in C, Perl, Python, Matlab or even in Verilog
- **The behavior of the circuit is compared against this golden model.**
  - Allows automated systems (very important)

GEBZE TEKNİK ÜNİVERSİTESİ 53

53

Testbenches

Why is Verification difficult?

- **How long would it take to test a 32-bit adder?**
  - In such an adder there are 64 inputs = 2^{64} possible inputs
  - That makes around 1.85 10^{19} possibilities
  - If you test one input in 1ns, you can test 10^9 inputs per second
    - or 8.64 x 10^{14} inputs per day
    - or 3.15 x 10^{17} inputs per year
  - we would still need **58.5 years** to test all possibilities
- **Brute force testing is not feasible for all circuits, we need alternatives**
  - Formal verification methods
  - Choosing 'critical cases'
  - Not an easy task

GEBZE TEKNİK ÜNİVERSİTESİ 54

54

Testbenches

What did we learn?

- **Verilog has other uses than modeling hardware**
  - It can be used for creating testbenches
- **Three main classes of testbenches**
  - Applying only inputs, manual observation (**not a good idea**)
  - Applying and checking results with inline code (**cumbersome**)
  - Using testvector files (**good for automatization**)

GEBZE TEKNİK ÜNİVERSİTESİ 55

55

Remarks

- **SystemVerilog** veya **VHDL** geniş bir dil, bütün ayrıntılarına girecek vaktimiz yok. Sadece işimize yarayacak kadarıyla uğraşacağız.
- Bir dili öğrendiğiniz zaman **diğer dile geçiş yapması** çok zor değil.
- Aynı programlama dilleri gibi HDL leri de **pratik** yaparak öğrenebilirsiniz.
- Her yazdığınız kodun bir **donanıma** dönüşeceğini unutmayın. O yüzden mutlaka **kontrol** edin.
- Artık devreleri HDL yardımıyla gösterebildiğimiz ve testbench yazabildiğimiz için lablarda bu yolu kullanacağız.
- ModelSim kullanmakla alakalı bir döküman paylaşıldı, labı yapmak için onu takip edin.

GEBZE TEKNİK ÜNİVERSİTESİ 56

56

END of LECTURE

- Thanks